



```

ET (EXP : TREE) return DEFSET_TYPE is begin if EXP.TY = DN_SELECTED then return FETCH_DEFSET (D (AS_DESIGNATOR, EXP)); else return
DEFSET_TYPE (CAST_SEQ_TYPE (D (SM_DEFN, EXP))); end if; end FETCH_DEFSET; procedure STASH_TYPESET (EXP : TREE; TYPESET : TYPESET_TYPE)
is begin D (SM_EXP_TYPE, EXP, CAST_TREE (SEQ_TYPE (TYPESET))); end STASH_TYPESET; function FETCH_TYPESET (EXP : TREE) return TYPESET_TYPE is
begin return TYPESET_TYPE (CAST_SEQ_TYPE (D (SM_EXP_TYPE, EXP))); end FETCH_TYPESET;
-----
procedure REDUCE_UNIQUE (ITEM_LIST : in out SEQ_TYPE; IS_CONFLICT : in out Boolean) is TEMP_LIST : SEQ_TYPE := IT
EM_LIST; TEMP_ITEM : TREE; TEMP_TYPE : TREE; -- MAY-BE DEF TOO RESULT_SEEN_WITHOUT_CONVERSION : Boolean := False; RESULT_SEEN_WITHOUT_CONFLICT : Boolean := False; RESULT_SEEN_WITH_CONFLICT : Boolean := False; function IS_CONFLICTING_CONVERSION (ITEM : T
REE) return Boolean is CONV_LIST : SEQ_TYPE; CONV_ITEM : TREE; TEMP_LIST : SEQ_TYPE; TEMP_ITEM : TREE; begin if ITEM
.TY /= DN_IMPLICIT_CONV then RESULT_SEEN_WITHOUT_CONVERSION := True; return False; end if; -- FAST RETURN IF ITEM WITHOUT
CONVERSION ALREADY SEEN if RESULT_SEEN_WITHOUT_CONVERSION then return True; end if; CONV_LIST := LIST (ITEM); if HEA
D (CONV_LIST) = TREE_VOID then return True; end if; while not IS_EMPTY (CONV_LIST) loop POP (CONV_LIST, CONV_ITEM);
TEMP_LIST := ITEM_LIST; while not IS_EMPTY (TEMP_LIST) loop POP (TEMP_LIST, TEMP_ITEM); if TEMP_ITEM = ITEM then null; elsif TEMP_IT
EM.TY /= DN_IMPLICIT_CONV then RESULT_SEEN_WITHOUT_CONVERSION := True; return True; elsif not IS_MEMBER (LIST (TEMP_ITEM), CONV_ITEM) then r
eturn True; end if; end loop; end loop; return False; end IS_CONFLICTING_CONVERSION; begin -- REDUCE_UNIQUE IS_CONFLIC
T := False; if IS_EMPTY (ITEM_LIST) then return; end if; TEMP_LIST := ITEM_LIST; while not IS_EMPTY (TEMP_LIST) loop POP
(TEMP_LIST, TEMP_ITEM); if TEMP_ITEM.TY = DN_IMPLICIT_CONV then TEMP_TYPE := D (XD_ITEM, TEMP_ITEM); else TEMP_TYPE := TE
MP_ITEM; end if; -- IF THERE IS AN INTERPRETATION AS A UNIVERSAL INTEGER OR REAL if TEMP_TYPE.TY = DN_UNIVERSAL_INTEGER or TEMP_TYP
E.TY = DN_UNIVERSAL_REAL then -- THIS IS ONLY POSSIBLE INTERPRETATION -- IF THIS INTERPRETATION REQUIRES A CONVERSION NOT REQUIRED --
0140 ... BY SOME OTHER INTERPRETATION if IS_CONFLICTING_CONVERSION (TEMP_ITEM) then -- ALL INTERPRETATIONS CONFLICT, SINCE OTHERS REQUIRE --
... CONVERSION OF THE UNIVERSAL TYPE ITEM_LIST := (TREE_NIL, TREE_NIL); IS_CONFLICT := True; -- ELSE -- SINCE THIS INTERPRETATION HAS NO C
ONFLICTS else -- THIS INTERPRETATION IS THE CORRECT ONE if TEMP_ITEM.TY = DN_IMPLICIT_CONV then ITEM_LIST := SINGLETON (D (XD_ITEM, T
EMP_ITEM)); else ITEM_LIST := SINGLETON (TEMP_ITEM); end if; IS_CONFLICT := False; end if; -- RETURN return; end if;
if IS_CONFLICTING_CONVERSION (TEMP_ITEM) then RESULT_SEEN_WITH_CONFLICT := True; else if RESULT_SEEN_WITHOUT_CONFLICT then
ITEM_LIST := (TREE_NIL, TREE_NIL); else ITEM_LIST := SINGLETON (TEMP_TYPE); RESULT_SEEN_WITHOUT_CONFLICT := True; end
if; end if; end loop; if RESULT_SEEN_WITHOUT_CONFLICT then return; else ITEM_LIST := (TREE_NIL, TREE_NIL); IS_C
ONFLICT := RESULT_SEEN_WITH_CONFLICT; end if; end REDUCE_UNIQUE; function COMBINE_EXTRAINFO (EXTRAINFO_1, EXTRAINFO_2 : EXTRAINFO_TYPE) retur
n EXTRAINFO_TYPE is -- GIVEN TWO EXTRAINFO LISTS, RETURN EXTRAINFO LIST WITH CONVERSIONS -- REQUIRED BY BOTH LISTS begin if IS_EMP
TY (SEQ_TYPE (EXTRAINFO_1)) then return EXTRAINFO_2; end if; if IS_EMPTY (SEQ_TYPE (EXTRAINFO_2)) then return EXTRAINFO_1; en
0150 d if; declare LIST_1 : SEQ_TYPE := SEQ_TYPE (EXTRAINFO_1); LIST_2 : SEQ_TYPE := SEQ_TYPE (EXTRAINFO_2); RESULT : SEQ_TYPE := (T
REE_NIL, TREE_NIL); begin if HEAD (LIST_2) = TREE_VOID then LIST_2 := TAIL (LIST_2); if HEAD (LIST_1) /= TREE_VOID then RES
ULT := SINGLETON (TREE_VOID); end if; if IS_EMPTY (LIST_2) then return EXTRAINFO_TYPE (APPEND (RESULT, LIST_1.FIRST)); end if
; end if; while not IS_EMPTY (LIST_1) loop RESULT := APPEND (RESULT, HEAD (LIST_1)); LIST_1 := TAIL (LIST_1); end l
oop; return EXTRAINFO_TYPE (APPEND (RESULT, LIST_2.FIRST)); end; end COMBINE_EXTRAINFO;
-----
end SET_UTIL;

```